

ARQUITECTURA Y GESTIÓN DE MEMORIA EN JAVA

**Ejemplos aplicados al sistema SISPE · Spring Boot,
Thymeleaf y MySQL**

Documento adaptado al proyecto real · 2026

Autor: Ing. William Ramón Flórez

1. Arquitectura base: JDK, JRE y JVM

SISPE se ejecuta sobre Java 17. Durante el desarrollo, el JDK compila las clases de `com.login.demo`, ejecuta Maven y permite depurar controladores como `MenuController` y `PedidoController`. En producción, el JRE/JVM carga Spring Boot, Thymeleaf, JPA y los controladores para atender las solicitudes HTTP. La JVM gestiona hilos para peticiones simultáneas de clientes, meseros y cocina.

2. Ciclo de vida de una aplicación SISPE

El código de `MenuController.java`, `UsuarioService.java` o `Pedido.java` se compila con Maven y Java 17 a bytecode `.class`. El `ClassLoader` carga esas clases; la JVM verifica su seguridad; Spring crea los beans mediante inyección de dependencias; y el compilador JIT optimiza métodos llamados frecuentemente, como listar productos o consultar pedidos. Una solicitud GET `/menu` se transforma en una respuesta Thymeleaf renderizada por la aplicación.

3. Stack y Heap en SISPE

En una petición a `/menu/guardar`, las variables locales del método `guardar` y sus referencias se alojan en el stack de cada hilo. El objeto `Menu` recibido por `@ModelAttribute`, la lista de categorías de `CategoriaRepository` y las entidades JPA viven en el heap. Al terminar la petición desaparecen los frames del stack; los objetos sin referencias quedan disponibles para el Garbage Collector. Un ciclo recursivo accidental puede producir `StackOverflowError` y cargar miles de productos en memoria puede producir `OutOfMemoryError`.

4. Objetos, referencias y ciclo de vida

Cuando el usuario agrega un producto al menú cliente, la aplicación crea referencias a objetos `Menu` y `Categoria`. Si un pedido mantiene una relación con `Mesa` y `Cliente`, esas referencias permiten recorrer el modelo JPA. Para liberar memoria, no se deben guardar listas completas de pedidos en objetos estáticos ni mantener sesiones con datos innecesarios. El carrito temporal debe contener solo los identificadores y cantidades necesarios.

5. Garbage Collector aplicado

El Garbage Collector puede recuperar objetos de respuestas Thymeleaf, DTOs, listas de categorías y resultados de consultas cuando ya no son alcanzables. Para favorecerlo, los servicios deben devolver colecciones limitadas, cerrar recursos correctamente y evitar caches globales sin límite. Las consultas de reportes deben filtrar por fechas, estado o mesa en MySQL en lugar de traer toda la tabla a la memoria de la JVM.

6. String Pool y textos del sistema

Textos constantes como "pendiente", "entregado", "Administrador" y mensajes de validación pueden reutilizarse en el String Pool. Para comparar estados se debe usar `equals` y no `==`. Por ejemplo, `pedido.getEstado().equals("pendiente")` compara contenido correctamente; usar `==` podría fallar aunque ambas variables contengan el mismo texto.

7. Autoboxing y colecciones

Los identificadores `Integer/Long` de `Menu`, `Mesa` y `Pedido` son objetos aunque sus valores representen números. Las colecciones como `List`, `List` y `Map` almacenan referencias y usan autoboxing al agregar valores primitivos. El equipo debe evitar crear objetos innecesarios dentro de bucles que recorren cientos de productos y debe validar null antes de convertir identificadores recibidos en formularios.

8. Memoria, concurrencia y sesiones

Cada solicitud HTTP puede ejecutarse en un hilo distinto. Los servicios SISPE deben ser stateless: no guardar datos de una petición en atributos mutables compartidos. Los datos temporales del cliente deben aislarse por sesión y las operaciones de guardar pedidos deben ser transaccionales. Las listas globales de pedidos o usuarios pueden provocar condiciones de carrera, fugas de memoria y exposición de información.

9. Buenas prácticas de optimización

Usar paginación para reportes grandes; seleccionar solo columnas necesarias; aplicar índices a claves foráneas y campos de búsqueda; liberar referencias temporales; evitar concatenar grandes Strings en bucles; y configurar límites razonables de pool de conexiones. En Thymeleaf se deben enviar al modelo solo productos y categorías necesarios para la vista. La optimización debe medirse con tiempos P95 y consumo de heap, no hacerse por suposición.

10. Conclusión aplicada

Comprender JDK, JVM, stack, heap, referencias, String Pool, autoboxing y Garbage Collector permite mantener SISPE estable bajo múltiples pedidos simultáneos. Los ejemplos muestran cómo la teoría de memoria se relaciona con Menu, Categoría, Mesa, Pedido, servicios, controladores, sesiones y vistas Thymeleaf. La calidad final depende de combinar diseño limpio, consultas eficientes, pruebas y observabilidad.